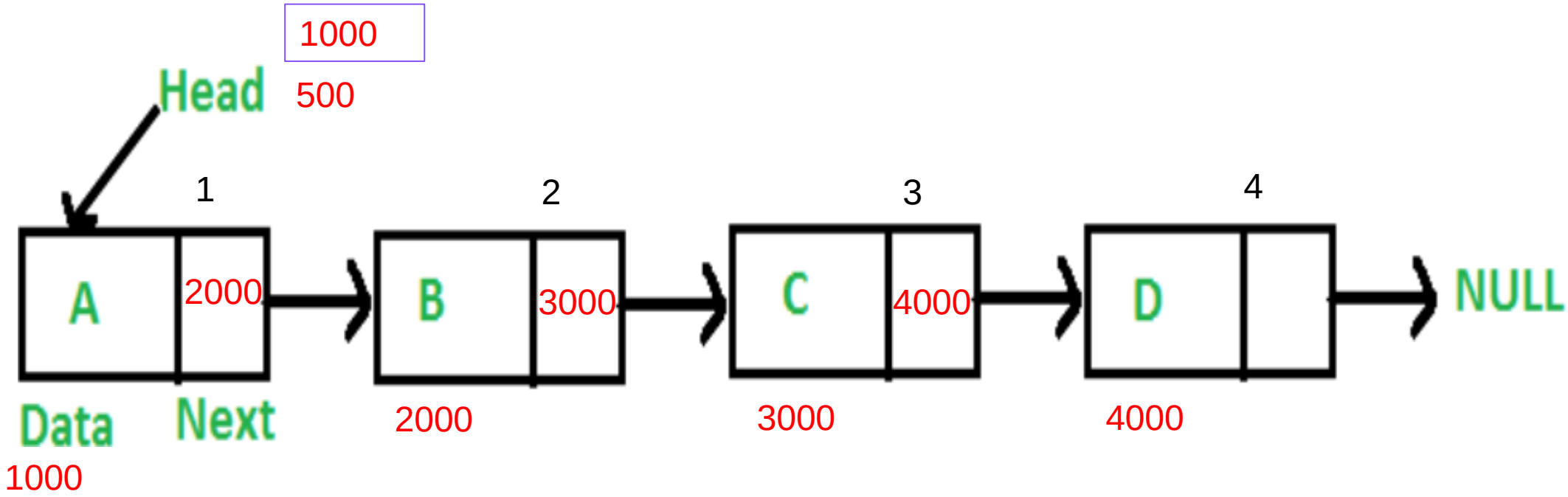

LINKED LIST
CREATION AND
TRAVERSING

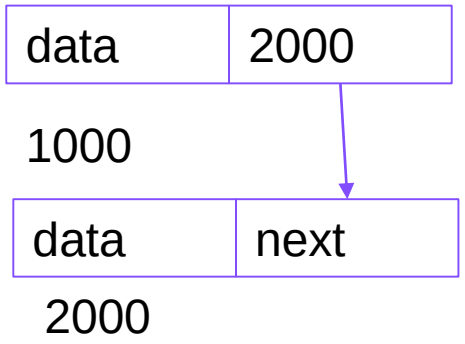
Linked Lists

- Unlike arrays, linked list elements are not stored at contiguous location, the elements are linked using pointers



Representation of a node in C

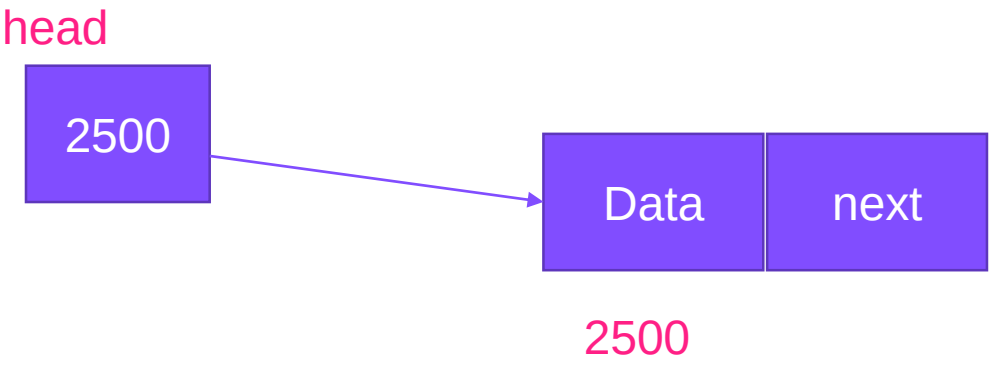
- Each node in a list consists of at least two parts: Data & Pointer to the Next node.



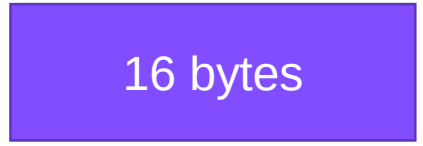

```
// A linked list node
struct Node
{
   int data;
   struct Node *next;
};
```

C program to create a linked list of three nodes

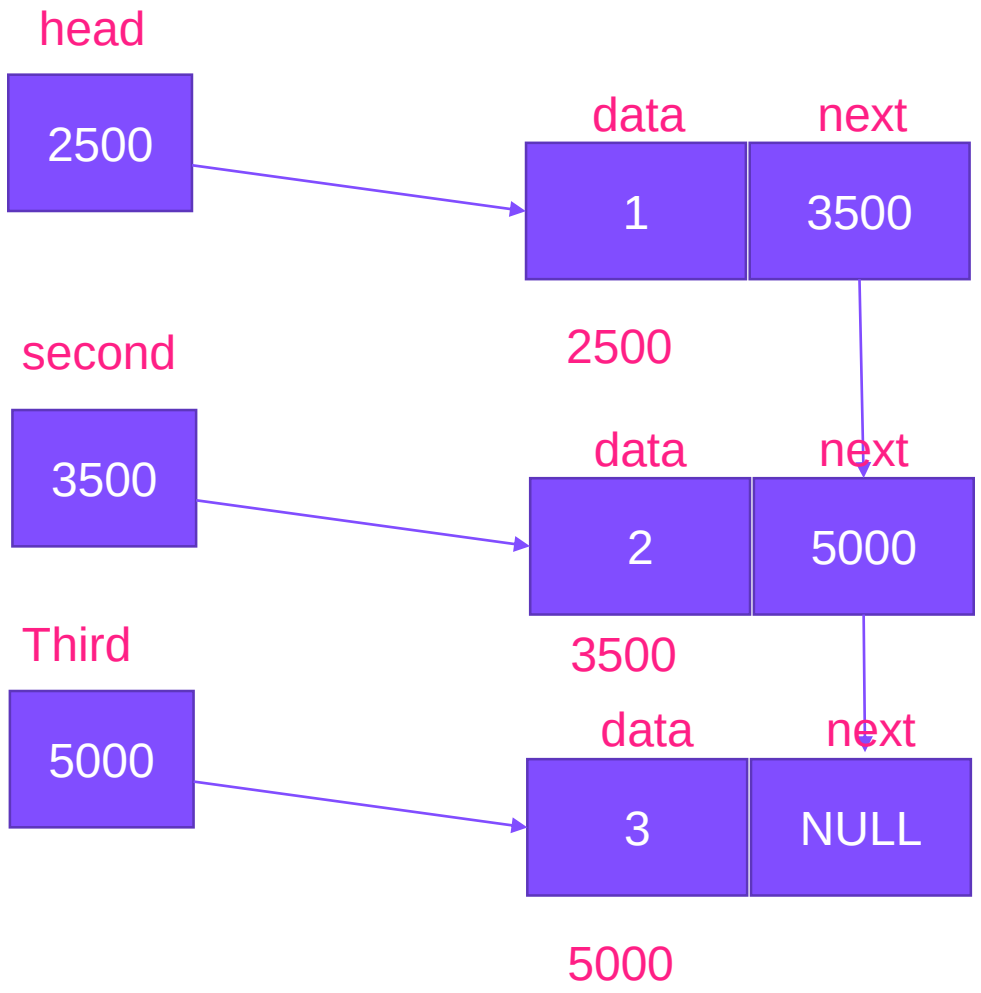
```c
// A simple C program to introduce
// a linked list
#include<stdio.h>
#include<stdlib.h>

struct Node
{
  int data;
  struct Node *next;
};

// Program to create a simple linked
// list with 3 nodes
int main()
{
  struct Node* head = NULL;
  struct Node* second = NULL;
  struct Node* third = NULL;

  // allocate 3 nodes in the heap
  head = (struct Node*)malloc(sizeof(struct Node));
  second = (struct Node*)malloc(sizeof(struct Node));
  third = (struct Node*)malloc(sizeof(struct Node));

  /* Three blocks have been allocated  dynamically.
     We have pointers to these three blocks as first, second and third
       head              second              third
        |                  |                  |
        |                  |                  |
    +---+-----+      +----+----+      +----+----+
    | #  | #  |      | #  | #  |      |  #  |  # |
    +---+-----+      +----+----+      +----+----+

   # represents any random value.
   Data is random because we haven't assigned anything yet  */
```



Int *ptr;
ptr=(int*)Malloc(sizeof(int*4))



Implementation



C program

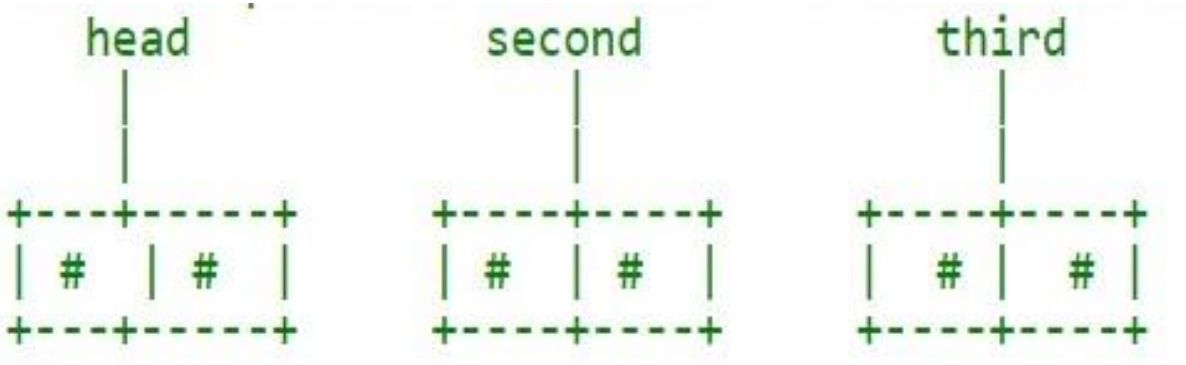


Fig 1.

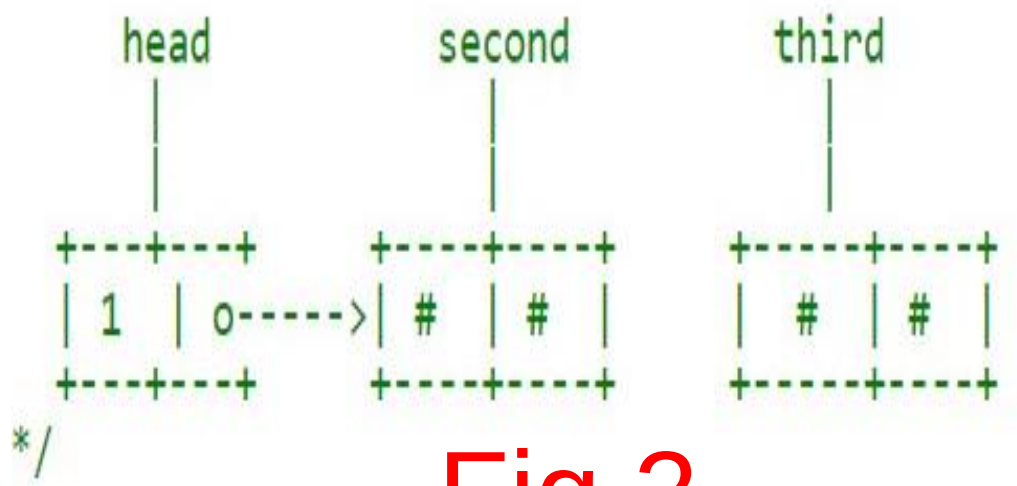
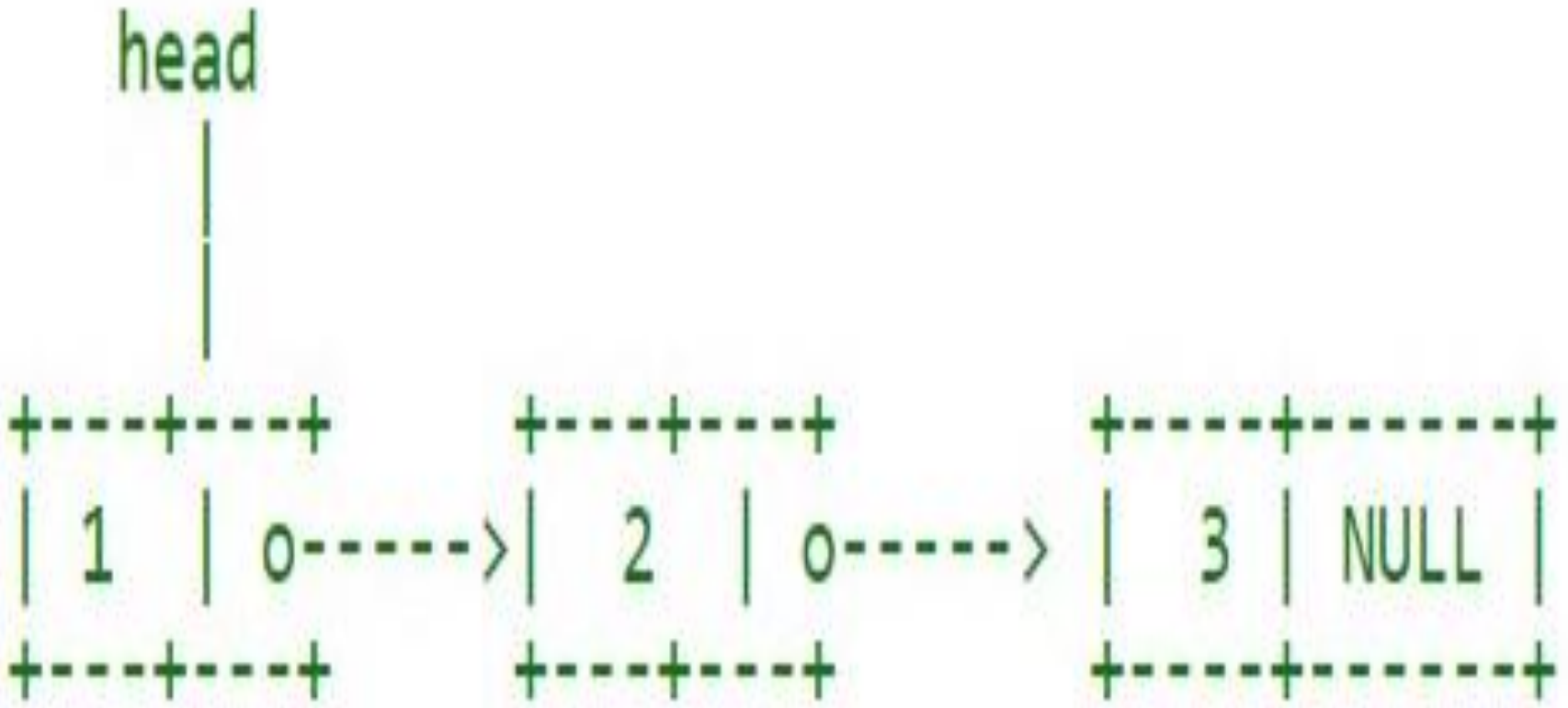


Fig 2.

head→ data =1;

head→ next=second;

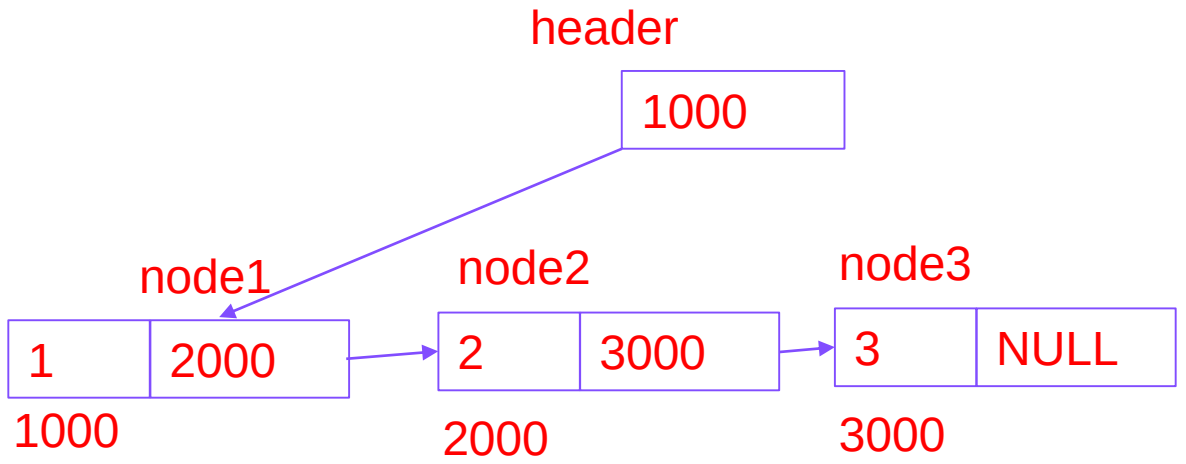

head
1
2
3
NULL

- second → data=2;
- second → next= third;

- third → data=3;
- third → next= NULL;

Traversing of linked list

- void traverse_list(struct Node *hptr)
- {
- while (hptr != NULL)
- {
- printf(“ %d”,hptr → data);
- hptr=hptr→next;
- }
- }



C code to create linked list with n nodes

```
node *create_list()
{
    int  k, n;
    node  *p, *head;

    printf  ("\n How many elements to enter?");
     scanf ("%d", &n);

    for  (k=0; k<n; k++)
    {
        if (k == 0) {
          head = (node *) malloc(sizeof(node));
          p = head;
       }
       else {
                p->next  = (node *) malloc(sizeof(node));
                p = p->next;
            }

        scanf ("%d", &p->data);
    }

    p->next  =  NULL;
    return (head);
}
```

To be called from `main()` function as:

```
struct node *head;
……
head = create_list();
traverse_list(head);
```

```
Struct node
{
  int data;
  struct node *link;
};
```

```
void traverse_list(struct Node
  *hptr)
{
   while ( hptr != NULL )
   {
    printf(" %d",hptr → data);
     hptr=hptr→next;
   }
}
```